

Evaluation of IPC through Linux Pipes

Patrick DeCabooter and Shaw Liu

1 Introduction

The UNIX pipe is the oldest IPC mechanism still in daily use, and its cost model is usually taken on faith: sixteen pages of buffer, writers block when it fills, larger writes amortize better. We measured those claims and then asked where the time goes.

The dominant variable turns out to be *placement*, not payload size. Two processes exchanging four bytes cost 708 ns per message on one core and 2042 ns across two, for identical work and identical system-call counts; the difference is entirely time spent descheduled. The same asymmetry governs bandwidth: the familiar throughput collapse at the pipe capacity happens only across cores. We establish capacity as its cause by varying it directly.

2 Apparatus and method

An Ubuntu 24.04 guest on a custom Linux 7.2.4 kernel (arm64), eight vCPUs, 12 GiB RAM, under QEMU/hvf on an Apple M1. Page size is 4 KiB and pipe-max-size is 1 MiB, so the resize sweep below runs unprivileged. Benchmarks are C at -O3 -Wall -Wextra. Absolute figures include hypervisor overhead; the comparisons between placements do not depend on it.

Processes are placed with sched_setaffinity after the fork, not with taskset, which imposes a single shared mask and cannot express “parent on core 0, child on core 1”. Each configuration is launched five times; each launch reports its internal minimum and we take the minimum again across launches, since noise can only add time. Throughput is a rate, so we report the maximum and never an arithmetic mean.

3 Choosing a clock

Variables. Cause: the clock API. Effects: accuracy, call cost, granularity. **Hypothesis.** clock_gettime can resolve a single exchange; gettimeofday cannot.

We compared clock_gettime(CLOCK_MONOTONIC), gettimeofday, and cntvct_el0, this architecture’s analogue of rdtsc. Timing sleep(5) five times, all three recovered 5.0024 s and agreed within 53 ppm; the 492 ppm excess is sleep’s own wakeup latency, which is precisely why the clocks agree on it.

Granularity came from 10^6 back-to-back reads into a preallocated array, post-processed afterwards, with nothing else in the loop (Table 1). The decisive column is the last: 97.9 % of adjacent gettimeofday readings are *identical*, two calls finishing inside one microsecond tick. A clock that cannot separate events by its own call cost cannot measure a 708 ns round trip, so we discarded it.

The cycle counter is cheapest to read but not finest: cntfrq_el0 reports 24 MHz, a 41.67 ns period. clock_gettime shows that same 41 ns quantum because current_clocksource is arch_sys_counter — the vDSO reads that very register and scales it.

We chose clock_gettime, and not for speed: it costs about twice what the raw counter does per call (20.2 ns against 10.4 ns), and that overhead is charged to every measurement. The grounds are portability and exactness. clock_gettime is one POSIX call that behaves identically on arm64 and x86-64, whereas the cycle counter needs per-architecture inline assembly and, on x86-64, a calibration step: there is no cntfrq_el0 equivalent, so the counter must be timed against CLOCK_MONOTONIC and divided, importing that clock’s error into every reading taken afterwards. On arm64 the rate is exact and reported by the hardware, but a clock we can trust on only one

Clock	Call (ns)	Quantum (ns)	Zero deltas	Predicted $1 - c/q$
clock_gettime	20.2	41.0	53.7 %	50.7 %
gettimeofday	22.8	1000.0	97.9 %	97.7 %
cntvct_el0	10.4	41.7	75.4 %	75.0 %

Table 1: Clock granularity over 10^6 consecutive reads. “Zero deltas” is the fraction of adjacent readings that were identical; the last column is what a call of cost c against a quantum q predicts. The agreement is the point: a high fraction measures a call that is cheap relative to one tick, not a clock that is coarse. Only `gettimeofday` is disqualified, and by its $1\mu\text{s}$ quantum rather than by its zero fraction.

of the two architectures our code compiles for is not one the result should rest on. `clock_gettime` additionally guarantees monotonicity, which a raw `rdtsc` read does not. Since the two share a quantum, taking the portable clock costs resolution nothing.

Neither standard cycle-counter caveat bites here: frequency invariance follows from the `sleep` agreement, and reading `cntvct_el0` on each of eight cores showed at most 83 ns of spread with no systematic step, the arm64 generic timer being system-wide rather than per-core.

4 Latency

Variables. Cause: payload size and placement. Effect: one-way latency, taken as half a round trip so only one clock on one core is involved. **Hypothesis.** Flat while fixed cost dominates, then linear once copying does.

Figure 1 confirms that shape and adds one the hypothesis missed. Below 4 KiB every curve is flat: cost is per exchange, not per byte. Above 16 KiB growth is linear, and the same-core slope implies 19.1 GiB/s of copy bandwidth. It is computed from the two endpoints of that curve, 708 ns at 4 B and 26,208 ns at 512 KiB, a span of 25,500 ns; the dashed line in Figure 1 is that model drawn over the data. A round trip copies the payload twice, once out of the writer and once into the reader, while the plotted value is half a round trip, so the two factors of two cancel and the bandwidth is simply payload over span.

The surprise is the constant offset in the flat re-

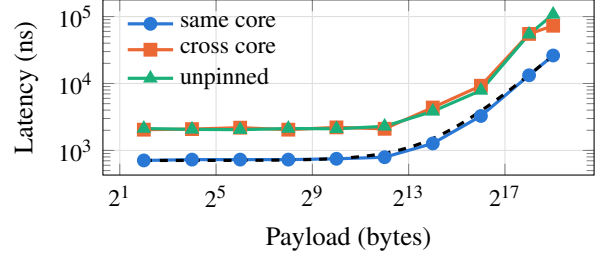


Figure 1: One-way latency (half round trip), minimum of five runs. The flat region below 4 KiB is fixed cost per exchange; the constant gap between same-core and cross-core is 1333 ns. The dashed line is the fixed-cost-plus-linear-copy model fitted to the same-core endpoints.

Payload (bytes)	unpinned	same core one-way latency (ns)	cross core
4	2125	708	2042
16	2062	729	2084
64	2042	729	2188
256	2125	729	2042
1024	2104	750	2208
4096	2292	792	2104
16384	3834	1270	4375
65536	7875	3270	9250
262144	54438	13271	55042
524288	107689	26208	72958

Table 2: One-way latency by payload and placement, minimum of five runs. The data behind Figure 1.

gion: 708 ns same-core against 2042 ns cross-core, a payload-independent penalty of 1333 ns. Unpinned runs track the cross-core curve, so the scheduler spreads the pair by default. Section 6 accounts for that penalty.

5 Throughput

Variables. Cause: chunk size and placement. Effect: sustained one-way bandwidth, acknowledged once for the whole transfer. **Hypothesis.** Throughput rises as the fixed per-write cost amortizes, then falls once chunks exceed capacity and the writer blocks.

Figure 2 shows the first half right and the second wrong. Only the cross-core curve collapses, peaking at 6431 MiB/s at 8 KiB and falling to 2391 MiB/s by 512 KiB. The same-core curve climbs to 9678 MiB/s

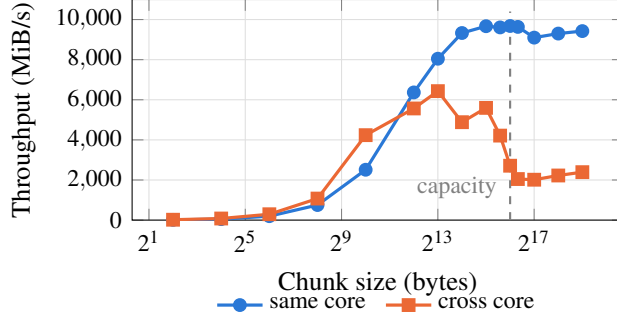


Figure 2: Streaming throughput at the default 64 KiB capacity (dashed line). Only the cross-core curve collapses; the same-core curve saturates and stays flat well past capacity, although its writer blocks just as often.

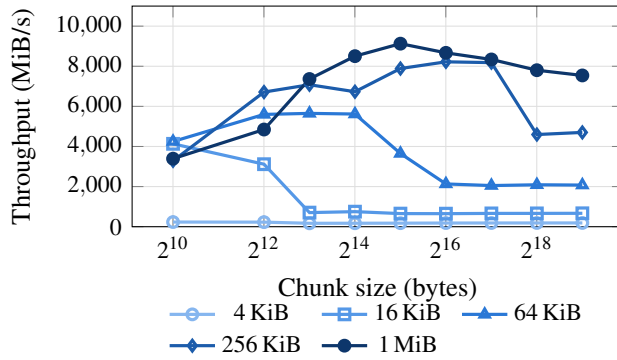


Figure 3: Cross-core throughput with the pipe capacity set by `F_SETPIPE_SZ`, one curve per capacity, light to dark. The knee tracks the configured size, which establishes capacity as the cause rather than a correlate.

and stays flat through chunks eight times the capacity. That falsifies blocking as the explanation: the same-core writer blocks just as often, the pipe being the same size, and pays nothing for it.

The mechanism is loss of *overlap*. Across cores, writer and reader truly run in parallel, and that pipelining survives only while the writer can deposit a whole chunk without stalling. Past capacity it stalls mid-chunk every time, the pair falls into lockstep, and every handoff costs a full cross-core wakeup. On one core there was no parallelism to lose, so bandwidth simply saturates at what the cache hierarchy sustains. The crossover at 4 KiB follows: below it overlap beats locality, above it locality wins.

Chunk (bytes)	unpinned	same core	cross core
	throughput (MiB/s)		
4	16	13	19
16	78	49	80
64	287	196	293
256	1095	752	1074
1024	4197	2511	4236
4096	5737	6366	5567
8192	6437	8048	6431
16384	5505	9329	4882
32768	4811	9668	5596
49152	4106	9608	4214
65536	3264	9678	2712
81920	3488	9634	2048
131072	1984	9099	2014
262144	1976	9301	2225
524288	2003	9425	2391

Table 3: Streaming throughput by chunk size and placement, best of five runs. The data behind Figure 2.

6 Where the time goes

Variables. Cause: placement and the ratio of payload to pipe capacity. Effects: the split of elapsed time between user, kernel and blocked, and the number of context switches and system calls per exchange. **Hypothesis.** If the cross-core penalty is a scheduling cost rather than a copying or system-call cost, then the two placements will issue identical system calls and identical context switches, and the whole difference will appear as time that is neither user nor kernel.

Filling a nonblocking pipe until `EAGAIN` accepts exactly 65,536 bytes, matching both `F_GETPIPE_SZ` and the sixteen-page folklore; single-byte writes reach the same total, so no page is wasted per write.

Table 4 accounts for the wall clock with `getrusage`. At 4 B the same-core pair spends essentially all its time in the kernel and none blocked. The cross-core pair does identical work — `strace` confirms exactly two write and two read calls per round trip — yet is ten times slower, with 61 % of its wall clock in neither user nor system time. That residue answers the 1333 ns question: it is time descheduled, awaiting a wakeup on another core.

The context-switch column closes the loop with capacity: 1.99 voluntary switches per round trip at 4 B and 15.99 at 512 KiB, exactly $2 \lceil \text{size}/\text{capacity} \rceil$ against the measured 64 KiB. `strace` gives the same arithmetic from the other side — at 512 KiB the

Payload	Place	$\mu\text{s}/\text{RT}$	%usr	%sys	%blk	cs/RT
4 B	same	2.3	0	101	0	1.99
4 B	cross	23.0	3	37	61	2.00
512 KiB	same	58.9	14	88	0	15.99
512 KiB	cross	346.8	0	53	47	15.99

Table 4: Where the wall clock goes, per round trip. Percentages are `getrusage` totals for both processes against elapsed time and may exceed 100 % by a point from tick granularity. “cs/RT” is voluntary context switches per round trip.

reader issues nine read calls per round trip, eight for data and one for the ack, because a read returns only what is buffered. Capacity sets the number of handoffs; placement sets the price of each.

Finally we made capacity the independent variable. Figure 3 re-runs the cross-core sweep with `F_SETPRIME_SZ` from 4 KiB to 1 MiB. The knee follows it: a 16 KiB pipe collapses at 8 KiB chunks, a 64 KiB pipe at 64 KiB, a 256 KiB pipe at 256 KiB, and a 1 MiB pipe, larger than any chunk tested, never collapses. The correlation is causal.

7 Conclusions

Pipe cost is set by two quantities, and payload size is only one of them. Capacity fixes how many handoffs a transfer needs, exactly $2\lceil \text{size}/\text{capacity} \rceil$ voluntary context switches, confirmed against a measured 64 KiB and then verified causally by moving it. Placement fixes the price of each handoff: 1333 ns of descheduled time when the peer is on another core, three quarters of the total at small payloads.

The practical consequence is counterintuitive. Spreading a communicating pair across cores buys parallelism worth having only while messages stay below capacity; past that, the pair is four times faster confined to a single core. A benchmark that leaves placement to the scheduler measures the cross-core case without saying so.

Two limits bound these results: everything ran virtualized, so absolute latencies carry hypervisor overhead, and no PMU was available in the guest, so Table 4 rests on `getrusage` and `strace` rather than hardware counters. We report the residue as blocked because it is neither user nor system time, not because a counter placed it there.

8 Use of AI tools

We used an AI coding assistant in three roles. It helped verify code revisions, reviewing the benchmark sources for defects and checking that changes behaved as intended. It helped conduct the experiments, driving the measurement sweeps across the placement and capacity matrices and aggregating the resulting CSVs into the tables and figures reported here. And we used it to brainstorm, particularly in working out which follow-up experiments would distinguish competing explanations for the throughput collapse.

The experimental design, the interpretation of the results and the claims made in this paper are our own, and we take responsibility for them. Every number reported here was produced by a committed benchmark on the machine described in Section 2 and is reproducible from the committed data.